



北京交通大学

电工电子实验教学中心

嵌入式系统课程设计

设计报告

基于二进制编码的声波通信系统的设计

学生一	姓名:	敖亚运
	学号:	21211175
	专业:	信息
学生二	姓名:	李科瑜
	学号:	21291172
	专业:	电子
选课教师:		郭薇薇

2024 年 10 月 30 日

目录

1 设计任务要求 1

2 系统设计方案 1

 2.1 系统顶层设计 1

 2.2 小组人员分工 2

 2.3 模块间的接口定义 2

3 功能模块设计方案 8

 3.1 发送端 CubeMX 配置 8

 3.2 显示模块 9

 3.3 编码发声模块 9

 3.4 译码接收模块 11

4 系统设计调试中的问题及解决方案 12

 4.1 重复采集同一个信息位 12

 4.2 起始频率无法稳定检测 12

 4.3 接收端无法准确同步起始位和信息位 12

5 系统使用说明 12

 5.1 系统外观及接口说明 12

 5.2 系统操作使用说明 12

 5.3 技术指标 13

6 总结 14

 6.1 设计结果 14

 6.2 心得体会 14

 6.3 意见建议 15

1 设计任务要求

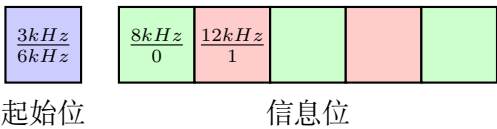
根据声波通信的工作原理，设计并制作一种具有利用声波进行控制信号传输功能的音响系统，该系统由声波通信主机和显示终端两部分组成，显示终端能够接收声波通信主机发送的信息并显示。

2 系统设计方案

2.1 系统顶层设计

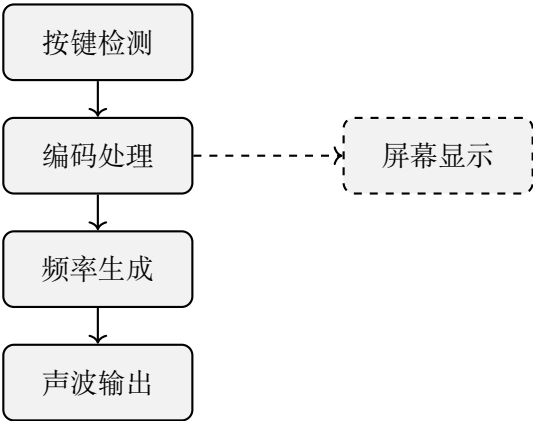
本次实验任务要求设计并制作一种通过声波进行控制信号传输的系统。系统分为两部分：声波通信主机和显示终端。声波通信主机通过声波传输编码信息，显示终端接收该信息并解码后进行显示。通信信息包含数字、颜色和 LED 开关控制。声波通信采用特定频率编码信息，通过边缘检测区分数据信号并解码。

系统编码方案采用了 6 位编码，由 1 位起始位和 5 位信息位组成，用于传输数字、颜色等级和 LED 开关控制三种信息。不同信息类型对应不同的起始位频率，信息位用特定频率表示 0 和 1。具体编码方式如下：



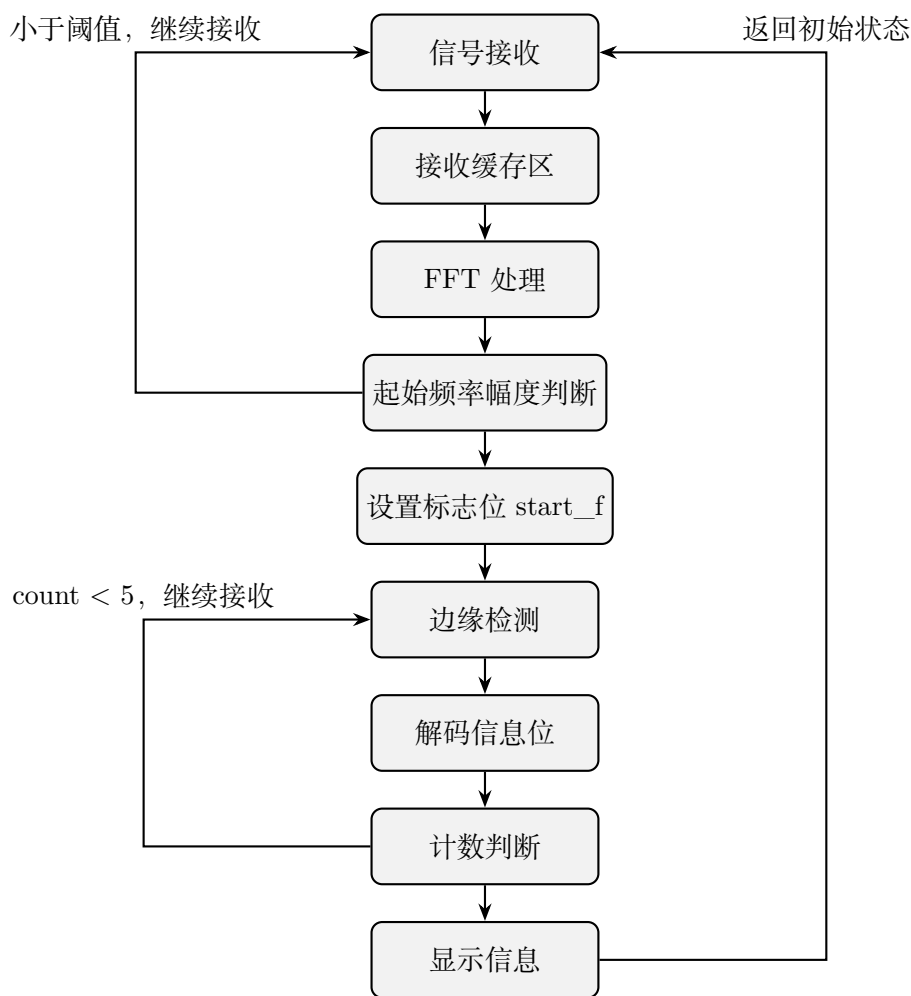
其中，起始位频率 3kHz 用于表示数字编码或 LED 控制编码，起始位频率 6kHz 用于表示颜色编码。信息位采用频率代表二进制位的方式进行传输,8kHz 表示信息位 0,16kHz 表示信息位 1。对于数字编码，5 位信息位编码方式如下，前 4 位表示数字 0-9 的二进制值，最后 1 位固定为 1，用于标识这是一个数字编码。对于 LED 控制编码，前 4 位用于指示 LED 的开关状态,最后 1 位固定为 0，用于区分 LED 信号和数字信号。对于颜色编码，前三位表示每种颜色的等级，最后两位表示颜色类型。

发送端通过按键输入、编码处理和声波信号生成，实现对数字、颜色和 LED 状态的声波传输。发送端的核心功能模块包括按键检测模块、编码模块、频率生成模块和用户界面模块，各模块协调工作以实现指定的控制信号编码和传输。



发送端运行逻辑系统框图

接收端接收并解码由发送端通过声波传输的控制信号。整个接收端由信号接收模块、FFT 处理模块、解码模块和显示模块组成，各模块协同工作，以实现对数字、颜色等级和 LED 控制信息的正确解码和显示。



接收端运行逻辑系统框图

2.2 小组人员分工

在本实验中，李科瑜和敖亚运各负责接收端和发送端的开发。敖亚运负责发送端的编码和声波生成。通过设计编码方案，他用不同频率表示起始位和信息位，并通过按键将输入转化为二进制编码存入数组 `ma[5]`，再由声波生成函数发出对应信号。他实现了屏幕显示功能，直观显示数字、RGB 颜色等级和 LED 状态，并提出了接收端的边缘检测方案，与李科瑜共同优化了编码方案。李科瑜完成了接收端的部分设计，包括音频信号接收、FFT 分析、标志位控制和信息位解码存储。他通过配置 SAI2 模块和缓存区实现音频接收，使用 FFT 提取频率幅值以识别起始信号，并设计了接收数组与计数器，确保稳定接收 5 位信息。李科瑜还设计了 UI 界面，实时显示接收的数字、颜色和 LED 状态。

2.3 模块间的接口定义

硬件接口

喇叭接开发板的 JP3 SPK R 接口。

接收端

```
//WindowDLG.c
//全局变量
#define ID_TEXT_3      (GUI_ID_USER + 0x04) // 数字显示句柄
#define ID_TEXT_7      (GUI_ID_USER + 0x08) // 红色等级显示
#define ID_TEXT_9      (GUI_ID_USER + 0x0A) // 绿色等级显示
#define ID_TEXT_11     (GUI_ID_USER + 0x0C) // 蓝色等级显示
#define ID_TEXT_13     (GUI_ID_USER + 0x0E) // LED 开关状态显示
#define ID_GRAPH_0     (GUI_ID_USER + 0x0F) // 红色基色显示
#define ID_GRAPH_1     (GUI_ID_USER + 0x10) // 绿色基色显示
#define ID_GRAPH_2     (GUI_ID_USER + 0x11) // 蓝色基色显示
//函数
//CreateWindow(void) 中 return hWin, 返回窗口的句柄供其他函数调用
//update_window_text (WM_HWIN hWin, const char* text, int n), 根据不同 n,
//调用不同的文字显示句柄,
//ID_TEXT_3、ID_TEXT_7、ID_TEXT_9 、ID_TEXT_11、ID_TEXT_13;
void update_window_text(WM_HWIN hWin, const char* text, int n)
{
    WM_HWIN hItem;

    // 使用窗口句柄获取文本控件的句柄
    if(n == 1)
        hItem = WM_GetDialogItem(hWin, ID_TEXT_3); //数字
    else if(n == 2)
        hItem = WM_GetDialogItem(hWin, ID_TEXT_7); //red
    else if(n == 3)
        hItem = WM_GetDialogItem(hWin, ID_TEXT_9); //green
    else if(n == 4)
        hItem = WM_GetDialogItem(hWin, ID_TEXT_11); // blue
    else if(n == 5)
        hItem = WM_GetDialogItem(hWin, ID_TEXT_13); //LED
    // 更新文本控件的内容
    TEXT_SetText(hItem, text);
}

// update_window_graph(WM_HWIN hWin, GUI_COLOR color, int mode):
//根据不同 n, 调用不同的颜色显示句柄: ID_GRAPH_0、ID_GRAPH_1、ID_GRAPH_2;
void update_window_graph(WM_HWIN hWin, GUI_COLOR color, int mode)
{
    WM_HWIN hGraph;
    if(mode==0){
        hGraph = WM_GetDialogItem(hWin, ID_GRAPH_0);
```

```
        GRAPH_SetColor(hGraph, color,0);
    }else if(mode==1){
        hGraph = WM_GetDialogItem(hWin, ID_GRAPH_1);
        GRAPH_SetColor(hGraph, color,0);
    }else if(mode==2){
        hGraph = WM_GetDialogItem(hWin, ID_GRAPH_2);
        GRAPH_SetColor(hGraph, color,0);
    }
}

//GUI_App.c:
//函数
//WM_HWIN hWin = CreateWindow();
//取得窗口的句柄
//arm_rfft32_app(hWin);
//传入函数 arm_rfft32_app 中
void GRAPHICS_MainTask(void) {
    WM_HWIN hWin = CreateWindow();
    while(1)
    {
        if(audio_capture_cplt)
        {
            arm_rfft32_app(hWin);
            audio_capture_cplt=0;
        }
    }
}

//main.c:
//全局变量
int bit_count = 0; //码元计数
char buffer[50]; // 显示文本
GUI_COLOR color=0xff000000; // 显示颜色
//函数
arm_rfft32_app(WM_HWIN hWin)//调用频率分析函数:
void arm_rfft32_app(WM_HWIN hWin)
{
    // uint16_t i;
    arm_rfft_fast_f32(&S, audio_input_F, audio_FFT_output_f32, 0);
    arm_cmplx_mag_f32(audio_FFT_output_f32,audio_FFT_output_mag_f32,INPUT_BUFSIZE/2);
    // 进行实时频谱分析
    analyze_frequency_component(hWin);
}

//analyze_frequency_component(WM_HWIN hWin) 中调用显示函数:
//总颜色显示改变:
```

```
DrawCircle(WM_HWIN hWin, GUI_COLOR color, int x, int y, int radius)
//引入窗口句柄, 显示颜色 color, 设置圆的参数, 基色颜色显示改变:
update_window_graph(WM_HWIN hWin, GUI_COLOR color, int mode)
//引入窗口句柄, 显示颜色, 设置不同 mode 改变不同位置的颜色模块
//文本显示改变:
update_window_text(WM_HWIN hWin, const char* text, int n)
//窗口句柄, 显示文本 buffer, 设置不同 n 改变不同位置的文本模块
//计算颜色等级
num = received_bits[0] + received_bits[1]*2 + received_bits[2]*4 +1;
//显示颜色等级
sprintf(buffer, "%d", num);
update_window_text(hWin, buffer,3);
//显示总颜色显示
color = (color & 0xffff00ff) + 0x00008f00 + 0x00001000*(num-1);
DrawCircle(hWin,color, 150, 130, 25);
//基色显示
update_window_graph(hWin, 0xff008f00+0x00001000*received_bits[0],1);
```

发送端

```
//main.c
//函数
//check(void), 检测屏幕被点击坐标, 存入 GUI 的位置存储中
void check(void)
{
    uint8_t buf[6];
    HAL_I2C_Mem_Read(&hi2c3, 0x70,0x02,1, buf, 1, 1000);
    if(buf[0]>0 && buf[0]<6)
    {
        HAL_I2C_Mem_Read(&hi2c3, 0x70,0x03,1, buf, 4, 1000);
        GUI_TOUCH_StoreState((buf[2]<<8|buf[3])&0x0fff,(buf[0]<<8|buf[1])&0x0fff);
    }
    else
    {
        GUI_TOUCH_StoreState(-1,-1);
    }
}

//audio_out(int* ma, int mode) 发声函数, 根据产生码元 ma 和 mode 发出对应声波,
//mode 对应不同模式产生不同起始位
if(mode == 1){
    BSP_AUDIO_OUT_Play( (uint16_t*)audio_output_start2, BUFSIZE*2 ); // 数字
    HAL_Delay(100)
    BSP_AUDIO_OUT_Stop(CODEC_PDWN_SW);
    HAL_Delay(100);
```

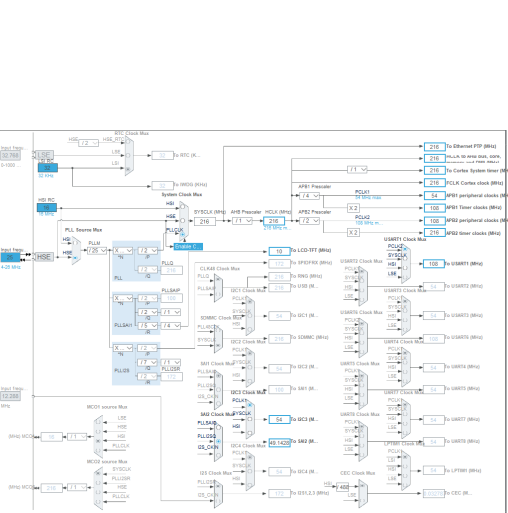
```
}
for ( int i=0 ; i < 5 ; i++ )
{
    if( ma[i] == 0 )
    {
        BSP_AUDIO_OUT_Play( (uint16_t*)audio_output2, BUFSIZE*2 ); // 发 0
        HAL_Delay(80);
        BSP_AUDIO_OUT_Stop(CODEC_PDWN_SW);
        HAL_Delay(80);
    }
    else if ( ma[i] == 1 )
    {
        BSP_AUDIO_OUT_Play( (uint16_t*)audio_output1, BUFSIZE*2 ); // 发 1
        HAL_Delay(80);
        BSP_AUDIO_OUT_Stop(CODEC_PDWN_SW);
        HAL_Delay(80);
    }
}
}
//GUI.App.c
//函数
//循环检测屏幕被点击的位置:
while(1)
{
    check();
    GUI_Delay(100);
}
//WindowDLG.c
//全局变量
#define ID_BUTTON_X 各个按钮组件
#define ID_TEXT_X    各个文本组件
#define ID_SLIDER_X  各个滑杆组件
#define ID_GRAPH_X   各个图标组件
uint32_t COLOR = 0xff000000 ; // 显示颜色
int      f_color[3] = {0,0,0}; // 三基色等级存储
int      ma[5] = {0,0,0,0,0}; // 生成编码
//函数
//在不同组件的操作函数中导入发声函数和改变显示
//按钮按下:
intToBinary(0, ma); //将十进制转化为二进制在 ma 中存储
ma[4] = 1;          //最后一位表示产生的是数字的编码
audio_out(ma,1);    //发声函数, ma 为编码结果, 1 表示对应改变数字的起始位
sprintf(buf,"number: 1");
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_3); //改变数字显示
```



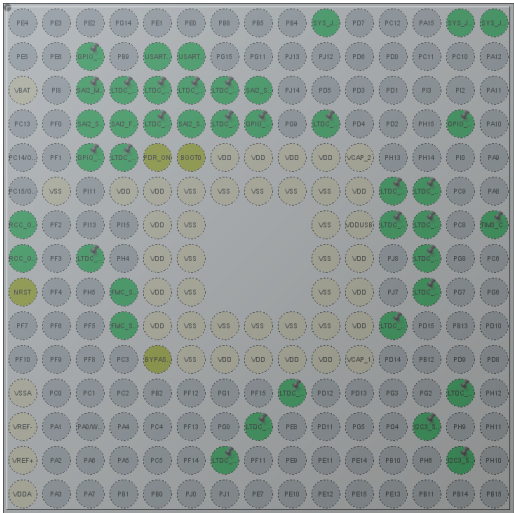
```
TEXT_SetText(hItem, buf);
//滑杆滑动 (部分代码)
hItem = WM_GetDialogItem(pMsg->hWin, ID_SLIDER_0);
int sliderValue = SLIDER_GetValue(hItem);
//根据滑杆值执行相应操作
if (sliderValue < 11) {
//执行操作 1
sprintf(buf, "red: %d", 0);
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0); //改变改色的显示等级
TEXT_SetText(hItem, buf);
hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0); //改变改色的基色显示
GRAPH_SetColor(hItem, 0xff000000,0);
COLOR = COLOR & 0xff00ffff;
hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_3); //改变总颜色的显示
GRAPH_SetColor(hItem, COLOR ,0);
f_color[0] = 0; //保存该颜色的等级状态
} else if (sliderValue < 22) {
// 执行操作 2 //与上同理, 对应等级不同
sprintf(buf, "red: %d", 1);
hItem = WM_GetDialogItem(pMsg->hWin, ID_TEXT_0);
TEXT_SetText(hItem, buf);
hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_0);
GRAPH_SetColor(hItem, 0xff8f0000,0);
COLOR = (COLOR & 0xff00ffff) + 0x008f0000; // color
hItem = WM_GetDialogItem(pMsg->hWin, ID_GRAPH_3);
GRAPH_SetColor(hItem, COLOR ,0);
f_color[0] = 1;
...
//编码部分:
if(f_color[0] > 0){ //判断颜色等级
intToBinary(f_color[0]-1, ma); //the level of red //转化为二进制
ma[3] = 0;
ma[4] = 1; // red //对应改变颜色
audio_out(ma,0); // 发声函数
}
```

3 功能模块设计方案

3.1 发送端 CubeMX 配置



(a) 发送端整体时钟树配置



(b) 整体管脚图

▼ Synchronization for Width

Horizontal Synchronization Width	1 pixels
Horizontal Back Porch	43 pixels
Active Width	480 pixels
Horizontal Front Porch	8 pixels
HSync Width	0
Accumulated Horizontal Back Porch Width	43
Accumulated Active Width	523
Total Width	531

▼ Synchronization for Height

Vertical Synchronization Height	10 lines
Vertical Back Porch	12 lines
Active Height	272 lines
Vertical Front Porch	4 lines
VSsync Height	9
Accumulated Vertical Back Porch Height	21
Accumulated Active Height	293
Total Height	297

▼ Signal Polarity

Horizontal Synchronization Polarity	Active Low
Vertical Synchronization Polarity	Active Low
Not Data Enable Polarity	Active Low
Pixel Clock Polarity	Normal Input

▼ Background Color

Red	0
Green	0
Blue	0

▼ Windows Position

Layer 0 - Window Horizontal Start	0
Layer 0 - Window Horizontal Stop	480
Layer 0 - Window Vertical Start	0
Layer 0 - Window Vertical Stop	272

▼ Pixel Parameters

Layer 0 - Pixel Format	RGB565
------------------------	--------

▼ Blending

Layer 0 - Alpha constant for blending	255
Layer 0 - Default Alpha value	0
Layer 0 - Blending Factor1	Alpha constant
Layer 0 - Blending Factor2	Alpha constant

▼ Frame Buffer

Layer 0 - Color Frame Buffer Start Address	0x20010400
Layer 0 - Color Frame Buffer Line Length (Image ...	480
Layer 0 - Color Frame Buffer Number of Lines (Im...	272

▼ Background Color

Layer 0 - Blue	0
Layer 0 - Green	0
Layer 0 - Red	0

▼ Number of Layers

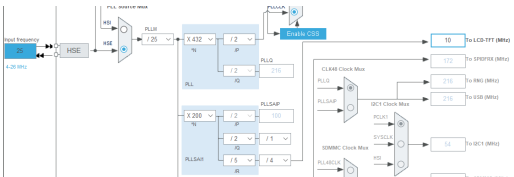
Number of Layers	1 layer
------------------	---------

(c) LTDC 显示屏参数设置

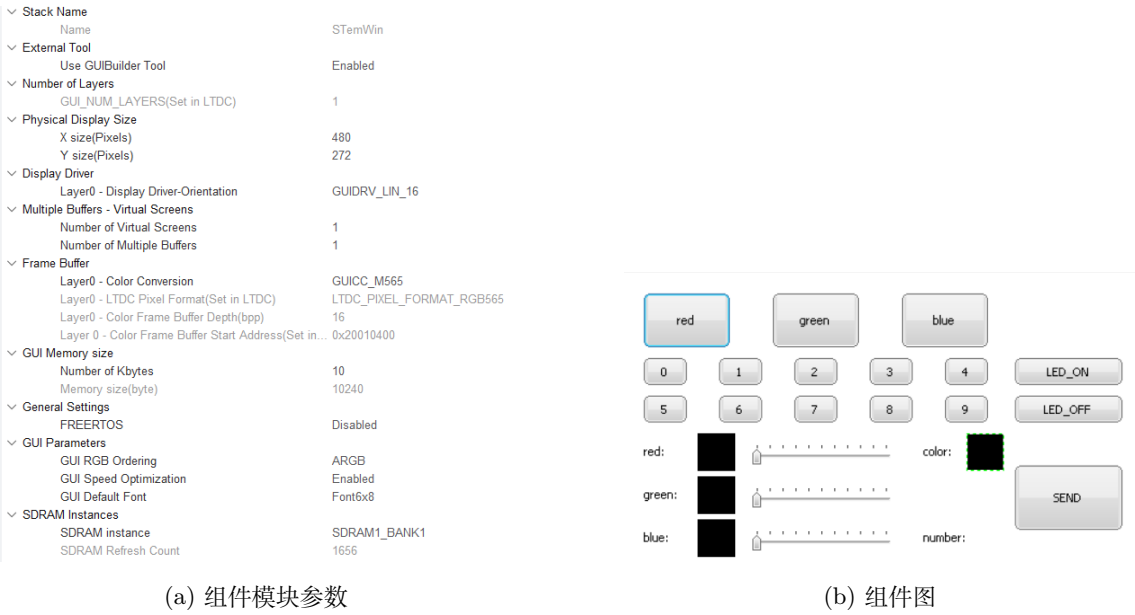
(d) LTDC 显示屏图层设置

Pin Name	Signal on Pin	GPIO output	GPIO mode	GPIO Pull-up	Maximum ou...	User Label	Modified
PG12	LTDC_B4	n/a	Alternate Fu...	No pull-up an...	Low		☐
PG19	LTDC_VSYNC	n/a	Alternate Fu...	No pull-up an...	Low		☐
PH10	LTDC_HSYNC	n/a	Alternate Fu...	No pull-up an...	Low		☐
PH14	LTDC_CLK	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ2	LTDC_R3	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ3	LTDC_R4	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ4	LTDC_R5	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ5	LTDC_R6	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ6	LTDC_R7	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ9	LTDC_G2	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ10	LTDC_G3	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ11	LTDC_G4	n/a	Alternate Fu...	No pull-up an...	Low		☐
PJ15	LTDC_B3	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK0	LTDC_G5	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK1	LTDC_G6	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK2	LTDC_G7	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK4	LTDC_B5	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK5	LTDC_B6	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK6	LTDC_B7	n/a	Alternate Fu...	No pull-up an...	Low		☐
PK7	LTDC_DE	n/a	Alternate Fu...	No pull-up an...	Low		☐

(e) LTDC 显示屏管脚配置



(f) LTDC 显示屏对应时钟树配置



3.2 显示模块

显示模块主要由 emwin 开放工具来进行显示设置，通过添加不同的组件来对应不同的操作，同时合理考虑所需功能与布局，兼顾美观、便于操作来进行设计。

Led 部分: 分为两个按键，一个负责 led 的开，另一个负责关

颜色部分

三基色对应不同按键，显示对应三个 graph 组件，总颜色对应一个 graph 组件，点击控制颜色的开关，也会在对应部分显示基色，叠加后的颜色也会显示在总颜色部分

全色彩通过滑杆设置不同等级进行显示操作，显示在对应的 graph 组件，叠加后的颜色显示在总颜色的 graph 组件，一个发送按钮将设置好的信息发出

数字部分：用十个按键对应十个数字，直观且便于操作

3.3 编码发声模块

采用板载的扬声器模块,WM8994 全功能音频处理器芯片挂载在 STM32F746 的 SAI2 接口上，通过软件设置初始参数，以及设置数字正弦信号，即可利用音频处理器芯片驱动喇叭发出想要频率的波形。之后配置 SAI2 及其对应管脚：

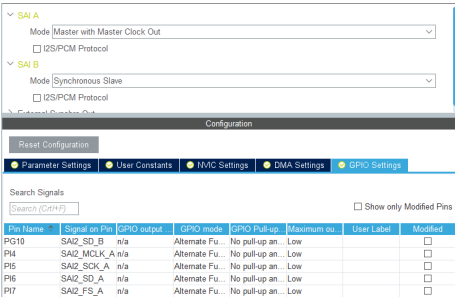


图 3: SAI2 及其对应管脚

对应时钟数配置：

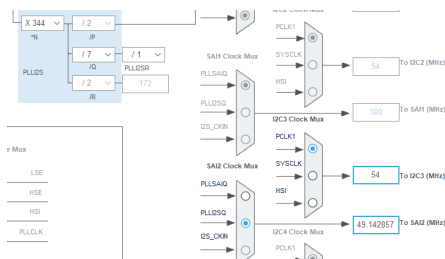


图 4: 对应时钟数配置

首先要确定所需频率范围，根据检测噪声频率以及方案所需频率数量，找到 3k、6k、8k、12k 四种频率，设置采样率 48kHz，可得到对应不同频率都能满足其周期所需的 inputsize 192；在不同的组件任务下，进行相对应的编码操作：Led 部分：起始频率 3kHz，编码最后一位为 0，led 开对应 11110 编码，关对应 00000 编码；数字部分：起始频率 3kHz，编码最后一位为 1，根据按钮对应的数字转换为二进制存入编码数组的前四位；颜色部分：

起始频率为 6kHz，最后两位代表不同的操作模式：00：基色模式，前三位表示对应基色的开关；01：改变红色，前三位表示改色的等级，范围：1-8；10：改变绿色，前三位表示改色的等级，范围：1-8；11：改变蓝色，前三位表示改色的等级，范围：1-8；通过相关判断产生对应编码 0、1，之后将编码数组与对应模式送入发声函数发送声波。

用板载的 led 灯来实现亮灭的操作, 在操作组件添加

```
HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1,GPIO_PIN_SET);
HAL_GPIO_WritePin(GPIOI, GPIO_PIN_1,GPIO_PIN_RESET);
```

函数来控制 led 的开关。

译码接收模块采用板载麦克分模块:

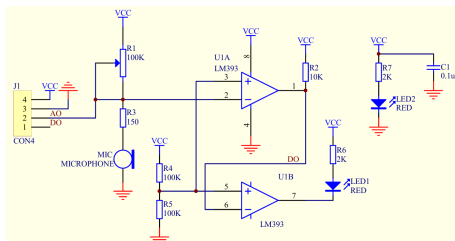


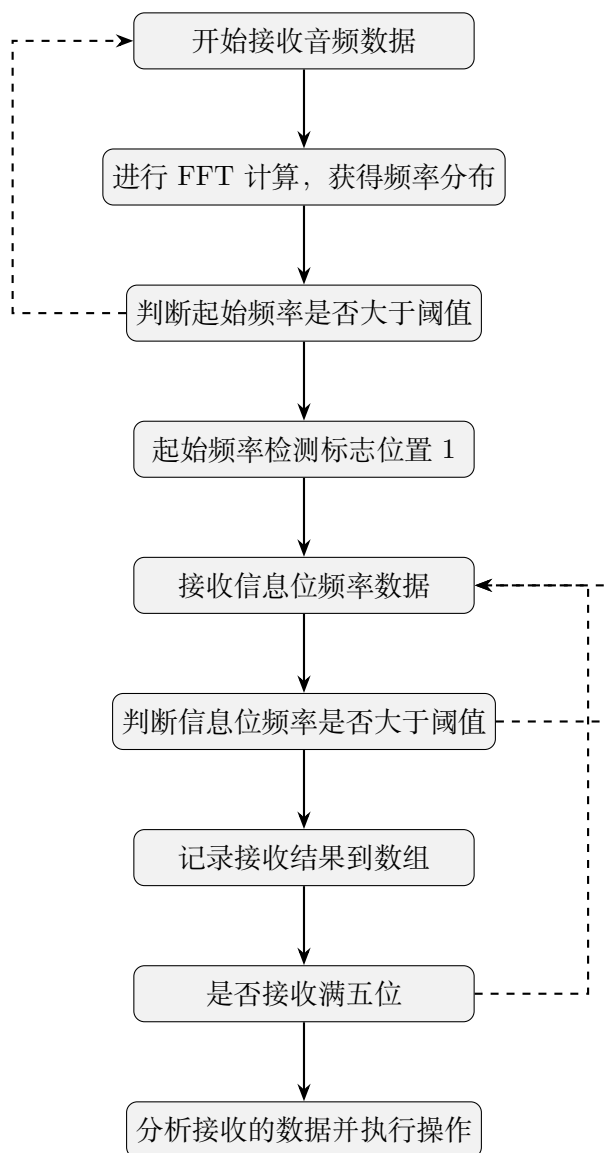
图 5: 板载麦克分模块

3.4 译码接收模块

接收端做 fft 得到频率分布，对频率进行分析，与所设阈值进行判断后执行相关操作。

设计过程：

每次 fft 后，取出起始频率的幅值进行判断，大于阈值后进行下一步操作，反正继续接收；接收到起始频率后，对应起始标志位置一表示已接收到起始频率，之后接收信息位；取出接收到的频率幅值，与对应阈值判断，大于则接收，8kHz 对应 1，12kHz 对应 0 将接收结果保存到数组中；接收满五位，根据发送端的编码设计分析结果，再根据起始标志位来进行对应的操作，标志位对应数字 led 和颜色模式；数字：将前四位的二进制转化为十进制，改变屏幕数字显示；Led：前四位为一进行 led 开操作，反之进行 led 关的操作；颜色：判断最后两位选择对应模式，再分析前三位，得出结果后改变颜色显示；



4 系统设计调试中的问题及解决方案

4.1 重复采集同一个信息位

问题描述：在接收端采集信息位时，系统总是重复采集同一个信息位的内容，导致解码结果错误，信息位无法准确解析。

原因分析：由于声波信号的频率在传播过程中会产生持续振荡，导致采样过程中会多次检测到同一个频率，从而重复采集同一信息位。

解决方法：采用边缘检测的方法，在上升沿计数来判断每个信息位的开始。当检测到上升沿后进入一个短暂的“抑制”状态，在该状态持续内不再进行采样检测，确保每个信息位仅被采集一次。这一方法显著减少了重复采样的问题，保证了解码的准确性。

4.2 起始频率无法稳定检测

问题描述：接收端在判断信号起始位频率（3kHz 或 6kHz）时，经常检测不到稳定的频率，导致无法正确识别信号类型。

原因分析：接收端起始频率的幅值较低，未达到设定的阈值，或者噪声影响了起始频率的判断，导致无法稳定捕捉信号的开始。

解决方法：通过多次测试和调整，优化了接收端的信号检测灵敏度，并在接收缓存区中累计多个采样周期的数据，以提高起始位的频率判断精度。同时，适当降低阈值并增加平均幅值计算，有效滤除了噪声影响，提升了起始频率检测的稳定性。

4.3 接收端无法准确同步起始位和信息位

问题描述：接收端偶尔出现无法同步起始位和信息位的现象，导致整个信号序列解析失败。

原因分析：由于信号的传播延迟或硬件响应时间导致起始位和信息位不同步，接收端在解码时出现位错。

解决方法：在接收端加入了起始位校准机制，通过多次检测起始频率来确保信号的开始，只有在连续检测到起始频率达到阈值的情况下才会开始接收信息位。此外，通过对接收缓存区的数据进行时间窗划分，有效解决了起始位与信息位不同步的问题，保证了信号解码的连续性和完整性。

5 系统使用说明

5.1 系统外观及接口说明

这里展示了显示界面控件和音频输入输出接口的位置和功能。

5.2 系统操作使用说明

在发送端，用户可以通过触摸屏界面操作。首先，用户可以点击数字按钮选择要发送的数字，点击后数字会在屏幕上显示，并通过声波发送至接收端。其次，用户可以控制 LED 的开关状态，点击“LED-ON”按钮开启 LED，点击“LED-OFF”按钮关闭 LED，屏幕将显示 LED 的当前状态。对于颜色选择，用户可以调整红、绿、蓝三种颜色的等级（1-8 级），不同

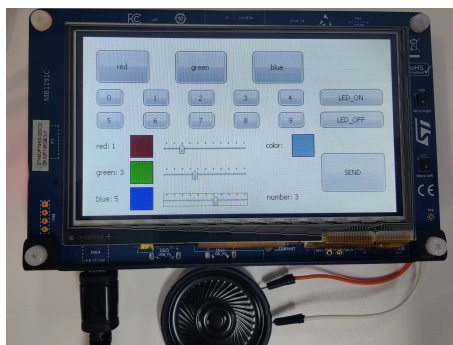


图 6: 发送端实物照片

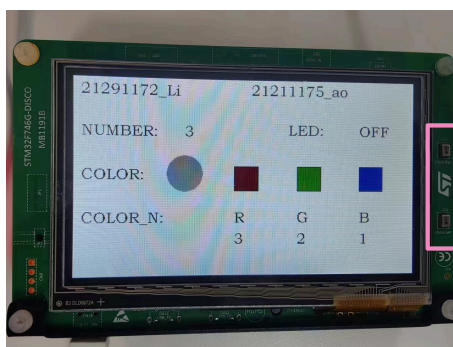


图 7: 接收端实物照片, 图中框住的部分为开发板的麦克风接口

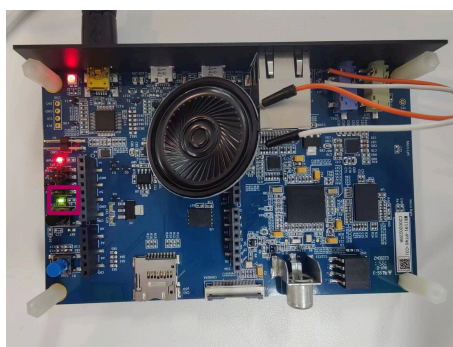


图 8: 开发板背面实物照片, 图中框住的部分为开发板的 LED 模块

等级代表颜色的深浅度。选择完成后，屏幕上将显示当前的颜色组合，并通过声波发送至接收端。所有选择完成后，点击“SEND”按钮即可将编码后的信号通过声波发送。

在接收端，系统会自动接收并解码声波信号，将其转换为对应的显示内容。当接收到数字信号时，屏幕会显示对应的数字。当接收到 LED 开关信号时，LED 会按信号指示进行开关操作，同时在屏幕上更新 LED 状态。当接收到颜色信号时，屏幕上将显示解码后的 RGB 颜色值和相应颜色。

整个系统通过简洁直观的 UI 界面，便于用户选择和发送数字、颜色以及控制 LED 状态。

5.3 技术指标

处理器：STM32F746NG 微控制器，主频 216 MHz，Cortex-M7 内核。

屏幕：4.3 英寸 LCD 显示屏，分辨率 480x272，支持触摸输入。

通信方式：通过声波传输控制信号，基于频率编码进行信息传输。

编码方式：

使用 6 位编码：1 位起始位 + 5 位信息位。

起始位 3 kHz 用于数字和 LED 控制信号，6 kHz 用于颜色编码信号。

数字信号编码范围为 0-9。

LED 控制信号分为开 (1111) 和关 (0111)。

颜色编码范围为 RGB 各 8 级，总共 512 种颜色。

频率设置：

信息位 “0” 使用 8 kHz，“1” 使用 16 kHz。

起始位频率：3 kHz（数字和 LED 控制）或 6 kHz（颜色编码）。

采样频率：系统采样率为 48 kHz，采样点数为 512 点。

解码方式：采用 FFT 频谱分析，检测特定频率幅值以解码信号，阈值设定用于滤除背景噪声。

误码率：由于采用边缘检测和幅度阈值判断，系统在较为安静环境下的误码率低于 5%。

传输距离：在较为安静环境中，传输距离可达 0.5 米，取决于声波信号强度和环境噪声。

响应时间：平均解码响应时间约为 700 ms，主要受 FFT 处理和声波传输影响。

LED 控制：内置 LED 支持远程开关控制，响应时间小于 200ms。

UI 界面：响应流畅，用户可以通过屏幕操作数字、颜色选择和 LED 控制，界面设计清晰直观，适合人机交互。

6 总结

6.1 设计结果

本次设计实现了基于 STM32F746 Discovery 开发板的声波控制信号传输系统，包括发送端和接收端两部分，成功完成了利用声波传递控制信息的功能。系统通过编码不同频率（如 3kHz、6kHz、8kHz 和 12kHz）来表示数字、颜色和 LED 控制信号，并在接收端解码后显示。发送端通过按键输入将用户的控制信息编码为指定频率的声波信号，并配有用户界面展示当前状态。接收端则通过 FFT 处理分析接收的声波信号，将幅值与设定阈值进行比较后判断起始位，并利用边缘检测准确接收信息位。解码后的信息实时显示在接收端屏幕上，并通过 LED 的开关状态体现系统的交互性。

系统整体上结构清晰，模块间逻辑分明，通过多重条件判断和循环逻辑确保了传输过程的准确性和稳定性。测试结果表明，系统能够稳定接收和解码发送端传输的信号，数字、颜色和 LED 控制信息传递准确无误，符合设计预期，本次设计实现了声波控制信号的可靠传输。

6.2 心得体会

敖亚运

在本次声波通信系统的设计过程中，我深刻体会到了在嵌入式开发中处理实时信号的重要性和挑战。声波通信对频率、幅值以及时序的要求非常严格，这让我在调试过程中不断地优化编码和解码的逻辑，以提高信号传输的准确性。特别是接收端的 FFT 处理和边缘检测，这些环节对代码效率提出了极高的要求。通过反复测试和调试，我不仅加深了对 FFT 分析的理解，还学会了如何设定合理的阈值来提高信号判断的可靠性。这次项目锻炼了我对信号处理和实时系统优化方面的能力，让我深刻意识到细节处理对于系统整体性能的巨大影响。

李科瑜

在参与本次声波控制系统设计的过程中，我对系统设计的模块化思维有了更深的认识。从按键输入到声波生成，从接收缓存到信息解码，每个模块都紧密相连且需具备高容错性，这使得系统能有效应对复杂的实时信号处理。尤其是接收端的流程环路设计，通过多重条件判断和循环逻辑保证了信号传输的准确性和系统的稳定性。我体会到，在开发复杂的嵌入式系统时，不仅要掌握硬件和编程知识，还需具备良好的逻辑思维和严谨的调试能力。这次项目让我意识到，在追求系统性能的同时，必须确保每个模块之间的良好协同，这也是实现高效和稳定的系统的关键。

6.3 意见建议